

Devine Café: uso das redes de Petri em um sistema de máquina de café

Allyson Bruno de Freitas Fernandes
UFERSA

Pau dos Ferros, Brasil
allyson.fernandes@alunos.ufersa.edu.br

Andrey de Oliveira Sabino
UFERSA

Pau dos Ferros, Brasil
andrey.sabino@alunos.ufersa.edu.br

Geísa Morais Gabriel
UFERSA

Pau dos Ferros, Brasil
geisa.gabriel@alunos.ufersa.edu.br

Klebson Davi de Souza Magalhães
UFERSA

Pau dos Ferros, Brasil
klebson.magalhaes@alunos.ufersa.edu.br

Lívia Beatriz Maia de Lima
UFERSA

Pau dos Ferros, Brasil
livia.lima30332@alunos.ufersa.edu.br

Pedro Damião de Oliveira Luz
UFERSA

Pau dos Ferros, Brasil
pedro.luz@ufersa.edu.br

Resumo—Este artigo focaliza o processo de desenvolvimento de um sistema por meio do uso de métodos formais de engenharia de software utilizando redes de Petri. A aplicação desse modelo matemático incide sobre um sistema de máquina de café intitulado Devine Café, criado visto a necessidade de tornar o processo de pedidos em uma cafeteria menos complexo e mais ágil. Para tal, pretende-se evidenciar a relevância da aplicação de processos formais e também de técnicas de qualidade de software no referido sistema. Com isso, procura-se entender mais detalhadamente a abordagem utilizada para cada um dos mecanismos supracitados, assim como retratar a relevância de construir sistemas conforme os aspectos de produção determinados pela engenharia de software. O presente trabalho justifica-se por relacionar aspectos de apoio ao desenvolvimento de sistemas visando reduzir erros, melhorar a comunicação e facilitar a manutenção por meio da integração e automatização dos processos. Assim, este artigo representa uma pesquisa de tipologia teórica a partir da criação de documentos, manuais e relatórios sobre o Devine Café, assim como da perspectiva prática do desenvolvimento de software em linguagem de programação C#. Revela-se, por meio deste, a pertinência em aplicar estratégias de aprimoramento de software com o uso de conceitos formais para a modelagem, análise e verificação de sistemas. Sobre tal conjuntura, a aplicação das redes de Petri para o Devine Café garantiu maior precisão, confiabilidade e segurança em relação às funcionalidades atribuídas ao sistema.

Index Terms—Sistema de Máquina de Café, Métodos Formais, Redes de Petri, Qualidade de Software.

I. INTRODUÇÃO

As interações humano-máquina compreendem um processo realizado em etapas que podem ser divididas em vários níveis, dependendo do perfil do usuário [19]. A realização de pedidos em uma cafeteria, por exemplo, pode acarretar alguns problemas, como erros de comunicação provenientes de pedidos mal interpretados, atrasos no atendimento e limitações de personalização de pedidos com precisão. Logo, em um sistema de máquina de café, facilitar a realização de pedidos em uma cafeteria, por meio da automatização de processos, promove maior autonomia ao cliente e, por consequência, permite gerar maiores lucros em um cenário comercial.

A fim de justificar a viabilidade em volta da motivação para realizar o desenvolvimento do sistema, foi efetuada uma breve pesquisa voltada, principalmente, ao consumo do produto e à frequência dos consumidores em cafeterias. Nesse sentido, 29% dos brasileiros tomam mais de seis xícaras de café por dia, enquanto 46% preferem de três a cinco xícaras [1]. Além disso, a alta na frequência em cafeterias (51% em 2023) após a pandemia da Covid-19, causada pela possibilidade de interagir com as pessoas, qualidade e modo de preparo do café, também são fatores motivacionais para a criação desse tipo de sistema [1].

Visto a viabilidade do produto de software e tendo em vista o desenvolvimento de sistemas, é necessário que o programa seja usável e compreensível por humanos, bem como atue de forma confiável e funcione corretamente [24].

Pensando no desenvolvimento do sistema de máquina de café conforme a aplicação dos métodos formais, foi desenvolvido neste estudo o Devine Café. Tendo em vista a ocorrência de falhas humanas no gerenciamento de pedidos, realizou-se a automatização de processos conforme o uso de uma linguagem formal baseada em rede de Petri, uma ferramenta matemática gráfica que oferecem um ambiente propício para modelagem, análise e simulação de sistemas de eventos discretos [7].

Logo, a aplicação dos métodos formais para o Devine Café procura auxiliar a compreensão dos requisitos e a identificação precoce de inconsistências, a fim de minimizar a complexidade e assegurar a correta implementação do sistema. Por essa razão, no contexto da engenharia de software, os métodos formais têm sido um dos meios utilizados para aumentar a qualidade e produtividade de software [20].

II. FUNDAMENTAÇÃO TEÓRICA

Esta seção, possui como finalidade expor os conceitos necessários para o melhor entendimento do assunto abordado. Dessa forma, são apresentados os conceitos relacionados ao desenvolvimento de sistemas; logo após, define-se redes de Petri e métodos formais de software; e, por fim, são apresentadas as tecnologias e ferramentas utilizadas.

A. Desenvolvimento de Sistemas

Em um contexto onde a tecnologia está cada vez mais presente, cresce a busca por sistemas computacionais de qualidade, com bom desempenho, baixo custo e com poucas falhas [8]. Tais características motivam a aplicabilidade de métodos para avaliar esses sistemas [9].

No desenvolvimento de sistemas, um sistema Web é, normalmente, constituído por um conjunto de páginas Web e por uma base de dados [12]. Esses sistemas utilizam a arquitetura cliente-servidor. Neles, a comunicação do cliente no navegador ocorre por meio do endereço IP (*Internet Protocol*) do servidor, responsável por hospedar a estrutura de dados [17].

Os sistemas Web permitem que os usuários acessem serviços, recursos e informações por meio de interfaces concedidas pelos navegadores Web. O acesso e troca de informações ocorre usando protocolos de comunicação (HTTP/HTTPS) baseados em texto que permitem a transferência de informações entre clientes e servidores web na internet [13].

Com a criação das aplicações Web o padrão de arquitetura MVC (Model-View-Controller) se popularizou [16]. Nesse padrão de arquitetura, todas as requisições da aplicação são direcionadas para a camada *Controller*, que acessa a camada *Model* para processar a requisição. O resultado do processamento é exibido por meio da camada *View* [16].

Para o desenvolvimento de sistemas, a documentação de software é essencial por dois motivos principais, uma das razões é facilitar a comunicação no processo de desenvolvimento do projeto e outra para esclarecer o conhecimento do programa nas atividades de manutenção [10]. A documentação atua, portanto, como uma facilitadora, esclarecendo precisamente o comportamento esperado do programa, bem como os possíveis desvios de fluxo [18].

Nos sistemas, o emprego de testes também é relevante para a descoberta de inconsistências e validação dos requisitos do software [24]. Para tal, cria-se uma gama de condições que descrevem os comportamentos a serem testados no software a partir da análise das especificações do sistema, denominados casos de teste [21]. Dessa forma, o teste é a principal técnica utilizada para a qualidade do sistema, uma vez que contribui na avaliação, controle e confiança do software [24].

A partir do desenvolvimento de um sistema Web de pedidos para uma cafeteria, aplica-se a definição de métodos formais de software por meio da utilização de redes de Petri.

B. Métodos formais e Redes de Petri

A aplicação de métodos formais de software consiste em abordagens rigorosas para desenvolver e verificar sistemas, utilizando modelos matemáticos para descrever e analisar suas funcionalidades e comportamento. Com isso, é possível garantir que programas estão corretos ou apresentam certas propriedades [11]. Logo, os métodos formais podem ser utilizados como suporte à geração de bons conjuntos de casos de teste e até mesmo para orientar no encontro de soluções mais elegantes, simples e eficientes [11].

Entre os métodos de modelagem formal existentes, podem-se destacar as redes de Petri. Rede de Petri é um formalismo

matemático representado graficamente e usado para modelar e analisar sistemas que tenham atividades paralelas, concorrentes, assíncronas e não-determinísticas [9]. Os lugares e transições das redes de Petri são usados para modelar uma visão lógica de pontos dos sistemas [23].

A representação gráfica de uma rede de Petri básica é formada por dois componentes: uma transição (barra) e um lugar (círculo). As transições correspondem às ações realizadas pelo sistema e os lugares equivalem às variáveis de estado. Esses dois componentes são ligados entre si por meio de arcos dirigidos. Marcas (*tokens*) são informações atribuídas aos lugares, para representar a situação (estado) da rede em um determinado momento [14].

Para descrever o comportamento desejado para o sistema, realiza-se a especificação formal. Especificar formalmente um sistema, consiste na utilização de notações formais, baseadas em técnicas matemáticas e na lógica formal, permitindo reduzir erros e ambiguidades, podendo servir como guia para o desenvolvimento do sistema e como base para a verificação da sua correção [5].

C. Tecnologias e Ferramentas

É comum adotar ferramentas a fim de auxiliar o ciclo de vida do software para facilitar seu gerenciamento e execução, bem como para utilizar melhor os recursos e seguir prazos [4]. Para tanto, o PIPE2¹ é uma ferramenta de código aberto para criar e analisar redes Petri. Essa ferramenta possui análise avançada, tais como na simulação, que indica o número médio de *tokens* por lugar, resultando em informações mais completas de como a rede de Petri se comporta [15].

Para a realização dos testes, o Lighthouse² é uma ferramenta automatizada de código aberto incorporada ao DevTools³, criada para melhorar a qualidade das páginas da Web. Por meio da execução dos testes usando a ferramenta, é possível gerar um relatório sobre o desempenho da página web. O xUnit.net⁴, por sua vez, é uma ferramenta de teste de unidade, de código aberto, licenciado sob Apache 2. Os testes realizados pelo xUnit baseiam-se na comparação do resultado obtido ao se executar uma função com o resultado esperado [22].

Para a análise estática, o SonarQube IDE⁵ é um *plugin* para IDE gratuito e de código aberto responsável por encontrar e corrigir problemas de codificação. O SonarQube Cloud⁶, por sua vez, consiste em uma ferramenta baseada em nuvem para fluxos de trabalho de Integração Contínua e Entrega/Implantação Contínua (CI/CD), que agiliza o trabalho dos desenvolvedores, permitindo testar quaisquer alterações feitas no repositório e fazer a entrega das alterações em diversos ambientes (produção ou testes) baseado em regras definidas, sem ser necessário utilizar serviços ou ferramentas externas [6].

¹Disponível em: <https://pipe2.sourceforge.net/>

²Disponível em: <https://developer.chrome.com/docs/lighthouse/overview>

³Disponível em: <https://developer.chrome.com/docs/devtools/?hl=pt-br>

⁴Disponível em: <https://xunit.net/?tabs=cs>

⁵Disponível em: <https://www.sonarsource.com/products/sonarlint/>

⁶Disponível em: <https://www.sonarsource.com/products/sonarcloud/>

O uso do SonarQube também garante a qualidade do código e aumenta a produtividade na resolução de problemas. Isso ocorre devido à tecnologia possui suporte para mais de 20 linguagens e usa mais de 5000 regras de *Clean Code* - conjunto de boas práticas de programação que buscam produzir código de fácil leitura, manutenção e evolução [3] - específicas de linguagem que buscam identificar erros comuns de codificação, *bugs* e vulnerabilidades.

O GitHub⁷ é um repositório de hospedagem de serviços Git. O Git⁸, por sua vez, é um sistema de controle de versões bastante utilizado no desenvolvimento de software. Com o uso do Git e GitHub, os usuários conseguem compartilhar projetos de forma gerenciada. Nesse sentido, os desenvolvedores sincronizam as mudanças feitas no seu repositório privado com o repositório principal [2].

III. ABORDAGEM

O presente estudo adotou uma abordagem de natureza aplicada ao desenvolvimento de um sistema de máquina de café utilizando redes de Petri. Para tal, os resultados obtidos no alicerce da metodologia empregada são detalhados sobre a documentação criada para o sistema, incluindo a especificação formal, bem como na aplicação prática do desenvolvimento do software. O conjunto dos artefatos confeccionados para o Devine Café⁹, incluindo *design*, modelagem, especificação e testes, pode ser visualizado detalhadamente no repositório¹⁰ público do GitHub.

Para fins explicativos da abordagem do sistema, este estudo irá enfatizar o funcionamento do Devine Café, tomando como base o fluxo principal da rede de Petri criada. Vale ressaltar que os requisitos e as regras de negócio expressos na documentação produzida, assim como a modelagem do sistema por meio dos diagramas de caso de uso e de classes, contribuíram para a construção da rede e entendimento do sistema em sua totalidade.

Dessa forma, foi utilizado o editor gráfico PIPE para modelar, simular e exportar a rede de Petri criada. Para escrever a especificação formal completa, com conjuntos de lugares, transições, função de fluxo, pesos, marcação inicial e análise de propriedades, utilizou-se o editor de texto do Google (Google Docs).

Para o Devine Café, o objetivo da rede de Petri criada foi representar o comportamento dinâmico do sistema, desde o início do pedido até a entrega ou cancelamento, assegurando a correção do fluxo e a prevenção de *deadlocks*.

Em aspecto geral, o sistema permite ao usuário realizar um pedido de café por meio de um processo interativo, passando pelas etapas de seleção do tipo, tamanho, quantidade, personalização com ingredientes, exibição de valor, confirmação, preparação e entrega. Apresenta também comportamentos alternativos, como o de cancelamento, que só

pode ocorrer antes do início do preparo. Os fluxos alternativos, por sua vez, permitem confirmação imediata, alteração de itens, bloqueio por indisponibilidade, e ações de "voltar" em qualquer etapa.

Na construção da rede, a etapa inicial incluiu a seleção de café, leite, açúcar e ingredientes; exibição e cálculo de valor; confirmação ou cancelamento do pedido; fluxos alternativos; exceções; controle de tempo e personalizações.

Logo após, foram definidos os lugares (estados do sistema). Cada etapa lógica do pedido foi representada por um lugar (P), o que representaria um estado do sistema naquele determinado tempo de execução, funcionando com uma espécie de captura de tela. Exemplos disso foram o início do processo, o tipo de café escolhido, o pedido confirmado e o pedido entregue.

Na terceira etapa, ocorreu a definição das transições, representada pelas ações do usuário e do sistema, conforme os requisitos previamente definidos. Já nas conexões por arcos (F), foram definidos todos os pares ordenados que conectam as ações aos estados.

A marcação inicial da rede criada¹¹ define onde o *token* começa. No início do processo, essa marcação é igual a 1 e nos demais lugares inicia com zero. Para a modelagem também foram incluídos fluxos alternativos, além do fluxo principal (pedido → personalização → confirmação → preparo → entrega). Esses fluxos incluídos referem-se a alguns comportamentos que o sistema pode oferecer a partir de determinadas interações. São eles: fluxo de cancelamento com tempo limite, alteração do pedido antes da confirmação, detecção de item indisponível e visualização dos detalhes do pedido.

Por fim, efetuou-se a revalidação dos requisitos. Para isso, cada requisito funcional e regra de negócio foi cruzado com os elementos da rede. Os requisitos não aplicáveis à rede (como criptografia ou *logs*) foram documentados como fora do escopo de Petri e tratados a partir dos testes executados. Validou-se também a conformidade da rede no que diz respeito a aspectos de alcance. Nessa perspectiva, a rede permite alcançar todos os estados finais esperados; vivacidade, todas as transições são ativáveis em seus respectivos contextos; segurança, nenhum lugar pode conter múltiplos *tokens* simultaneamente; e, ausência de *deadlock*, o fluxo sempre segue até um estado final ou de erro.

Finalizado o processo de modelagem, ocorreu a transição para o desenvolvimento do sistema. Assim, na construção do *back-end* utilizou-se a linguagem de programação C# e para o *front-end* a linguagem de programação TypeScript em conjunto com a biblioteca React, diante de uma arquitetura cliente-servidor. O roteamento da aplicação foi implementado com a biblioteca *react-router-dom*, permitindo navegação entre páginas distintas do sistema Web. Para a persistência dos dados, foi utilizado o banco de dados PostgreSQL em conjunto com o Docker. A API foi desenvolvida utilizando a arquitetura MVC, com uma implementação que segue o princípio da separação de responsabilidades [16].

⁷Disponível em: <https://github.com/>

⁸Disponível em: <https://git-scm.com/>

⁹Disponível em: <https://maquina-de-cafe-blue.vercel.app/>

¹⁰Disponível em: <https://github.com/Allysonfreitas210695/MaquinaDeCafe.git>

¹¹Disponível em: <https://github.com/Allysonfreitas210695/MaquinaDeCafe/blob/main/Artefatos/Especificacao/Devinecafe.png>

Para garantir o controle de qualidade conforme a rede, foi elaborado um plano de teste que incluiu o escopo, metodologia e objetivos que guiaram a avaliação do sistema. Dessa forma, os testes estruturais de unidade foram realizados com a ferramenta xUnit. Foram realizados também testes exploratórios, a fim de identificar erros e comportamentos inesperados a partir da aprendizagem, experiência, conhecimento e observação do testador. Ademais, a fim de coletar métricas de desempenho, boas práticas e acessibilidade realizou-se testes de qualidade de página com lighthouse. Para a análise estática do código-fonte, foi utilizada a ferramenta SonarQube.

IV. CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

Este trabalho esteve estruturado sobre a necessidade de implementar modelos formais sobre softwares. Para isso, foi realizado o desenvolvimento de um sistema de máquina de café. Assim, concluiu-se que a aplicação de métodos formais direcionado para os processos de interações com usuário, escolha de bebidas e personalização de preferências garantem maior precisão, confiabilidade e segurança no funcionamento, uma vez que permitem identificar inconsistências no design do sistema antes que elas se manifestem no uso real.

O uso de ferramentas e o entendimento documentado sobre as funcionalidades do sistema também são formas de agilizar o processo de desenvolvimento e garantir uma melhor comunicação entre as partes envolvidas no projeto.

Como trabalhos futuros, espera-se aprimorar o sistema para a implementação em uma máquina de café real. Para isso, pretende-se aprimorar alguns artefatos criados, como os requisitos existentes, adicionar os requisitos futuros sugeridos e atualizar a especificação formal. Além disso, para o melhor funcionamento, espera-se refinar o sistema a partir da aplicação de mais técnicas de teste.

REFERÊNCIAS

- [1] ABIC. Indicadores da indústria de café, 2024.
- [2] AQUILES, A., AND FERREIRA, R. *Controlando versões com Git e Github*. São Paulo: Casa do Código, 2014.
- [3] AZEVEDO, B. A. A. S. Clean code e solid na construção da aplicação oratio: melhorando a manutenibilidade e escalabilidade.
- [4] BARBOSA, H. O., BONIFACIO, B. A., MENEZES, T. M., UEBEL, L. F., PIRES, F. B., AND NETO, A. F. Uma análise do uso de ferramentas em desenvolvimento distribuído de software para atualização da plataforma android. *WWW/INTERNET 2019* (2019), 39–46.
- [5] BARBOSA, R. D. M. Especificação formal de organizações de sistemas multiagentes.
- [6] BORGES, J. M. C. Pipeline ci/cd para automação e orquestração de redes.
- [7] CARGNIN, R. S., AND ROOS-FRANTZ, F. Revisão da literatura de simulação de sistemas de eventos discretos com foco na integração de aplicações. *III SFCT* (2015), 15.
- [8] DA COSTA AGUIAR, J. B., AND GRACIANO, F. C. Gestão da qualidade no desenvolvimento de produto de software. *Revista Interface Tecnológica* 18, 2 (2021), 773–783.
- [9] DE LIMA, J. W. S., FALCÃO, T. P., AND ANDRADE, E. Trydp: uma ferramenta para o aprendizado de modelagem de sistemas usando redes de petri. In *Simpósio Brasileiro de Educação em Computação (EDUCOMP)* (2021), SBC, pp. 362–370.
- [10] DE SOUZA, S. C. B., DAS NEVES, W. C. G., ANQUETIL, N., AND DE OLIVEIRA, K. M. Documentação essencial para manutenção de software ii. In *IV Workshop de Manutenção de Software Moderna (WMSWM)*, Porto de Galinhas, PE (2007).
- [11] DÉHARBE, D., MOREIRA, A. M., RIBEIRO, L., AND RODRIGUES, V. M. Introdução a métodos formais: Especificação, semântica e verificação de sistemas concorrentes. *Revista de Informatica Teórica e Aplicada. Porto Alegre. Vol. 7, n. 1 (set. 2000)*, p. 7-48 (2000).
- [12] DELAMARO, M. E., MALDONADO, J. C., AND JINO, M. *Introdução ao Teste de Software*. Elsevier, 2016.
- [13] FIELDING, R. T., GETTYS, J., MOGUL, J., FRYSTYK, H., MASINTER, L., LEACH, P., AND BERNERS-LEE, T. Hypertext transfer protocol – http/1.1. Tech. Rep. RFC 2616, Internet Engineering Task Force (IETF), June 1999. Obsoleted by RFC 7230–7235.
- [14] FRANCÊS, C. R. L. Introdução às redes de petri. *Laboratório de Computação Aplicada, Universidade Federal do Pará* (2003).
- [15] KREBS, D. É. Estudo comparativo das ferramentas pipe2, mercury tool e timenet baseadas em redes de petri. *IV SFCT 7* (2016), 5.
- [16] LUCIANO, J., AND ALVES, W. J. B. Padrão de arquitetura mvc: Model-view-controller. *EPeQ Fafibe 1*, 3a (2017), 102–107.
- [17] MARTHA, L. F. *Desenvolvimento de uma aplicação web para modelagem colaborativa*. PhD thesis, PUC-Rio, 2022.
- [18] MCCONNELL, S. *Code Complete: A Practical Handbook of Software Construction*, 2 ed. Microsoft Press, Redmond, WA, 2004.
- [19] NIELSEN, J. *Usability Engineering*. Morgan Kaufmann, San Francisco, 1993.
- [20] PIMENTA, A. Especificação formal de uma ferramenta de reutilização de especificações de requisitos.
- [21] PRESSMAN, R. S. *Engenharia de software*, 7 ed. AMGH, Porto Alegre, 2011.
- [22] SANTIN, C. E. Desenvolvimento guiado por testes e ferramentas xunit.
- [23] SILVA, A. N. D., LINS, F. A. A., SANTOS JÚNIOR, J. C., ROSA, N. S., QUENTAL, N. C., AND MACIEL, P. R. M. Avaliação de desempenho da composição de web services usando redes de petri.
- [24] SOMMERVILLE, I. *Engenharia de software*. Pearson Universidades, 2011.